Recitation 2

CS211 section 10-13

January 30, 2024

To-Do

1. Logistics
2. Nano
3. C basics

Logistics

- biweekly quizzes from last week was a typo, **there are no quizzes**
- We put up a guide on accessing the iLab via Windows PowerShell, Mac OS, and Linux under Canvas announcements.

Nano Overview

- Nano is an open-source text editor designed for UNIX-based systems.
  - Licensed under the GPL (GNU Public License).
- Lightweight CLI(Command Line Interface) text editor.
- Much easier to get into compared to Vim, Emacs, and Neovim.

How to use Nano?

- Reference sheet (You'll realistically use a few of these, but here they are).
- To create a file:
  - CD to the directory where you want to create the file.
  - If you want to create a txt file, use `nano filename.txt`.
    - You don't need to touch a new file.
- To edit:
  - Write whatever you need in the file.
  - To save, press `Ctrl + O` (`^O`).
  - To exit, press `Ctrl + X` (`^X`). If you didn't save, you'll be prompted to do so.
  - The bottom of the editor lists out the instructions.

Nano Question

So, we know that ˆ means control and M means alt. Given this and the instructions at the bottom of nano, how do we undo and redo changes?

Nano Question

So, we know that ˆ means control and M means alt. Given this and the instructions at the bottom of nano, how do we undo and redo changes?

- Alt U to undo
- Alt E to redo

While Loop in C

- `while` loop in C executes Code repeatedly as long as a specified condition is true.

```
while (condition) {
    //Code executed while
    // condition is true
}
```

- The `condition` is a boolean expression; the loop executes as long as it's true.

```
int count = 0;
while (count < 5) {
    printf("%d\n", count++);
}
```

- Iterates five times, printing "0" to "4".

For Loop in C

- `for` loop in C iterates a specific number of times.

```
for (init; condition; update) {
    //Code executed each iteration
}
```

- Continues while `condition` is true.
- `update` is executed after each iteration.

```
for (int i = 0; i < 5; i++)
    printf("%d\n", i);
```

- Iterates five times, printing "0" to "4".

If statements

```
if (condition) {
    //Code to execute
}
```

If `condition` is true, everything inside the statement will be executed.

```
if (A > 100) {
    printf("A is greater than $100\n");
}
```

Arrays

- In C, an array is a collection of elements of the same data type.
- Declared using the syntax: `data_type array_name[size];`
- Example: `int numbers[5];` declares an integer array named `numbers` with a size of 5.
- Access elements similar to Java: `array_name[index_number];`
- What does `int value = numbers[2];` do?

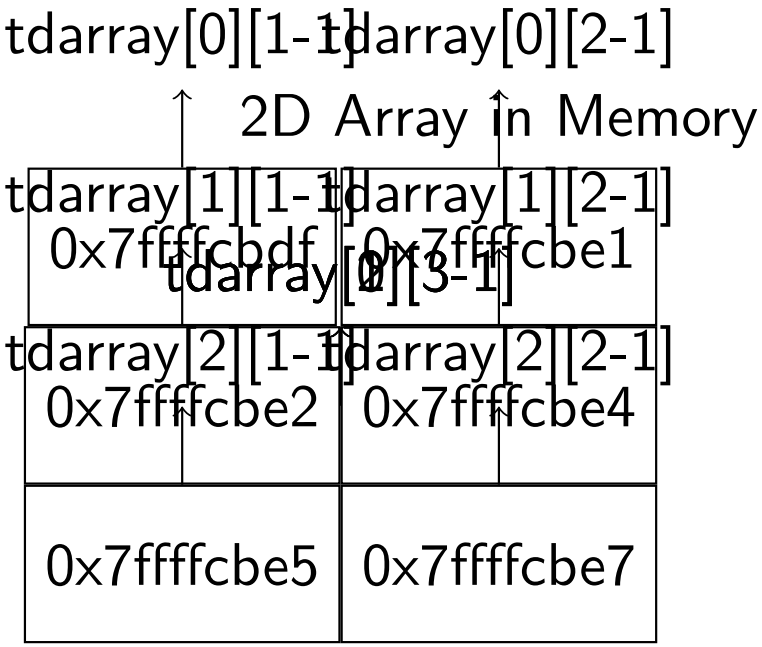
Arrays

- In C, an array is a collection of elements of the same data type.
- Declared using the syntax: `data_type array_name[size];`
- Example: `int numbers[5];` declares an integer array named `numbers` with a size of 5.
- Access elements similar to Java: `array_name[index_number];`
- What does `int value = numbers[2];` do?
- sets `value` equal to the third element of array.

2D Array

- In C, we can declare 2D arrays with the following: `data_type array_name[row size][col size]`
- We can access it using the same syntax as java.
- : What does this 2d array look like in memory?

2D Arrays - Continued



Switch Case

```
  GNU nano 6.2                                    switchcase.c
#include<stdio.h>

int main(int argc, char** argv){
    int a;
    scanf("%d",&a);
    switch(a){
    case 0: //if a is 0, print 0
        printf("0\n");
        break; //break is important, it leaves the switch case. Failure to use it will result in fall through behavior.
    case 1:
        printf("1\n");
        break;
    case 2:
        printf("2\n"); //we forgot break D-:
    default:
        printf("another number\n");
        break;
    }
    return 0;
}

                                                  [ Read 20 lines ]
```

Figure: Switch Case

Functions

Functions in C are weird.

```c
#include<stdio.h>
/**
* Functions follow the following format
* return_type function_name(arguments)
* for example: int GCD(int a, int b)
**/
//void printsum(int a, int b); //this is the declaration. We have put it here, or anywhere above the function call.
int getFive(){
    return 5;
}

void printsum(int a, int b); //We can put the declaration here instead if we want.

int main(int argc, char** argv){
    int x = getFive();
    printsum(4,5);
    return 0;
}
//void printsum(int a, int b); this will not help, the declaration must be placed above the function call.
void printsum(int a, int b){
    printf("a + b = %d",(a + b));
}
```

Figure: Functions in C

- If we call a function, and the function is below the call, we need to declare it.
  - printsum is below the call, so the compiler doesn't know what printsum is. The compiler will give an "implicit declaration of function" warning.
  - getFive is okay because it's above the function call at `int x = getFive()`

practice

```c
#include<stdio.h>

int main(int argc, char** argv){
    int a = 2342332;
    printxnum(a);
    return 0;
}


void printxnum(int a){
while(a > 0){
    printf("count: %d\n",a);
    a++;
}
}
```

Figure: Find the mistake in this Code.